

A Complete Reference with Examples & File Handling

Printable Edition · Covers NumPy · Pandas · Matplotlib · File I/O

Table of Contents

1. Introduction to Python for Data Analysis

2. Setting Up the Environment

- Installation
- Importing Libraries

3. NumPy – Numerical Computing

- Arrays
- Array Operations
- Indexing & Slicing
- Math Functions

4. Pandas – Data Manipulation

- Series & DataFrame
- Loading Data
- Exploring Data
- Filtering & Selecting
- GroupBy & Aggregation
- Merging & Joining

5. File Handling in Python

- Reading & Writing Text Files
- CSV Files
- JSON Files
- Excel Files
- Binary Files
- os & pathlib Modules

6. Data Cleaning

- Missing Values
- Duplicates
- Data Types

7. Matplotlib – Visualisation

- Line Plot
- Bar Chart
- Histogram
- Scatter Plot
- Subplots

8. Quick Reference Card

1. Introduction to Python for Data Analysis

Python has become the go-to language for data analysis, data science, and machine learning. Its rich ecosystem of libraries, readable syntax, and active community make it ideal for every stage of a data project—from ingestion and cleaning to visualisation and modelling.

Why Python for Data Analysis?

Feature	Benefit
Open Source & Free	Python and all major data libraries are free with no licensing costs.
Rich Ecosystem	NumPy, Pandas, Matplotlib, Scikit-learn, and hundreds more.
Readable Syntax	Code reads almost like English, making it easy to maintain.
Community & Resources	Massive community means solutions to problems are a search away.
Integration	Works seamlessly with databases, web APIs, cloud services, and more.

2. Setting Up the Environment

Installation

Install the core data-analysis libraries with pip:

```
# Install essential libraries
pip install numpy pandas matplotlib openpyxl

# For Jupyter notebooks (recommended for exploration)
pip install jupyterlab
```

Importing Libraries

By convention, libraries are imported with short aliases. Always place imports at the top of your script.

```
import numpy as np          # numerical arrays
import pandas as pd         # data manipulation
import matplotlib.pyplot as plt # plotting
import os                  # file system operations
from pathlib import Path    # modern path handling
import json                # JSON files
import csv                 # CSV files
```

3. NumPy – Numerical Computing

NumPy (Numerical Python) provides the `ndarray` object—a fast, memory-efficient multi-dimensional array—along with tools for mathematical operations, linear algebra, and random number generation.

Creating Arrays

```
import numpy as np

# 1-D array
a = np.array([10, 20, 30, 40, 50])
print(a)          # [10 20 30 40 50]

# 2-D array (matrix)
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.shape)    # (2, 3)

# Special arrays
zeros = np.zeros((3, 3))      # all zeros
ones  = np.ones((2, 4))      # all ones
eye   = np.eye(3)            # identity matrix
rng   = np.arange(0, 10, 2)   # [0 2 4 6 8]
lin   = np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1.]
```

Array Operations

```
a = np.array([1, 2, 3, 4])

print(a + 10)      # [11 12 13 14] - scalar broadcast
print(a * 2)       # [ 2  4  6  8]
print(a ** 2)      # [ 1  4  9 16]

b = np.array([10, 20, 30, 40])
print(a + b)       # [11 22 33 44] - element-wise
print(np.dot(a, b)) # 300          - dot product
```

Indexing & Slicing

```
arr = np.array([5, 10, 15, 20, 25, 30])

print(arr[0])      # 5    - first element
print(arr[-1])     # 30   - last element
print(arr[1:4])    # [10 15 20]
print(arr[::2])    # [ 5 15 25] - every other

# 2-D slicing
m = np.arange(1, 10).reshape(3, 3)
print(m[0, :])     # [1 2 3] - first row
print(m[:, 1])     # [2 5 8] - second column
print(m[1:, 1:])   # bottom-right 2x2 block
```

Statistical & Math Functions

```
data = np.array([4, 7, 2, 9, 1, 6, 8, 3, 5])

print(np.mean(data))    # 5.0
print(np.median(data))  # 5.0
print(np.std(data))     # 2.58...
print(np.min(data))     # 1
print(np.max(data))     # 9
print(np.sum(data))     # 45
print(np.sort(data))    # [1 2 3 4 5 6 7 8 9]
print(np.argmax(data))  # 3 (index of max value)
```

4. Pandas – Data Manipulation

Pandas provides two core structures: Series (1-D labelled array) and DataFrame (2-D table). It is the workhorse for loading, cleaning, transforming, and analysing structured data.

Series

```
import pandas as pd

s = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
print(s)
# a    100
# b    200
# c    300

print(s['b'])    # 200
print(s * 2)     # element-wise multiply
print(s.mean())  # 200.0
```

DataFrame

```
data = {
    'Name':  ['Alice', 'Bob', 'Carol', 'Dave'],
    'Age':   [25, 30, 35, 28],
    'Salary': [50000, 65000, 80000, 72000],
    'Dept':  ['HR', 'IT', 'IT', 'Finance']
}
df = pd.DataFrame(data)
print(df)
```

Exploring a DataFrame

```
df.head(3)      # first 3 rows
df.tail(2)      # last 2 rows
df.shape        # (4, 4) - rows x columns
df.columns      # Index(['Name', 'Age', 'Salary', 'Dept'])
df.dtypes       # data type of each column
df.info()       # summary: types, nulls, memory
df.describe()   # count, mean, std, min, max...
```

Filtering & Selecting

```
# Select columns
df['Name']       # one column (Series)
df[['Name', 'Salary']]  # multiple columns

# Filter rows
df[df['Age'] > 28]    # age > 28
df[df['Dept'] == 'IT']  # IT dept only
```

```
df[(df['Age'] > 25) & (df['Salary'] > 60000)]

# loc / iloc
df.loc[0]                # row by label
df.iloc[1:3]             # rows 1-2 by position
df.loc[df['Dept']=='IT', 'Salary'] # filtered column
```

GroupBy & Aggregation

```
# Average salary per department
df.groupby('Dept')['Salary'].mean()

# Multiple aggregations
df.groupby('Dept').agg(
    avg_salary=('Salary', 'mean'),
    headcount=('Name', 'count'),
    max_age=('Age', 'max')
)
```

Merging & Joining

```
dept_info = pd.DataFrame({
    'Dept':      ['HR', 'IT', 'Finance'],
    'Location':  ['Delhi', 'Mumbai', 'Bangalore']
})

# Inner join (only matching rows)
merged = pd.merge(df, dept_info, on='Dept', how='inner')

# Left join (keep all rows from left df)
merged = pd.merge(df, dept_info, on='Dept', how='left')
```


5. File Handling in Python

File handling is at the heart of every data pipeline. Python gives you built-in tools for plain text, plus library support for structured formats like CSV, JSON, and Excel. Understanding file I/O lets you load raw data, process it, and save results reliably.

5.1 Reading & Writing Plain Text Files

The built-in `open()` function is the foundation of file handling. Always use the `with` statement—it automatically closes the file even if an error occurs.

File Modes

Mode	Meaning	File must exist?
"r"	Read only (default)	Yes
"w"	Write – creates or overwrites	No
"a"	Append – adds to end of file	No
"x"	Create – fails if file exists	No
"r+"	Read + Write	Yes
"rb"	Read binary	Yes
"wb"	Write binary	No

Writing to a Text File

```
# Write multiple lines to a file
lines = ['Alice,25,HR', 'Bob,30,IT', 'Carol,35,IT']

with open('employees.txt', 'w') as f:
    f.write('Name,Age,Dept\n')    # header
    for line in lines:
        f.write(line + '\n')

print('File written successfully.')
```

Reading a Text File

```
# Method 1 – read entire file as one string
with open('employees.txt', 'r') as f:
    content = f.read()
    print(content)

# Method 2 – read line by line (memory-efficient for large files)
with open('employees.txt', 'r') as f:
    for line in f:
        print(line.strip())    # .strip() removes trailing \n

# Method 3 – read all lines into a list
with open('employees.txt', 'r') as f:
```

```
all_lines = f.readlines()    # ['Name,Age,Dept\n', ...]
```

Appending to a File

```
# Add a new employee without erasing existing data
with open('employees.txt', 'a') as f:
    f.write('Dave,28,Finance\n')
```

■ Always use `'with open(...)'` instead of `open() + close()`. If your script crashes, `'with'` guarantees the file handle is released.

5.2 CSV Files

CSV (Comma-Separated Values) is the most common format for exchanging tabular data. Python provides two approaches: the built-in csv module and Pandas.

Using the csv Module

```
import csv

# ■■ Writing CSV ■■
employees = [
    ['Name', 'Age', 'Salary', 'Dept'],
    ['Alice', 25, 50000, 'HR'],
    ['Bob', 30, 65000, 'IT'],
    ['Carol', 35, 80000, 'IT'],
]

with open('employees.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(employees)

# ■■ Reading CSV ■■
with open('employees.csv', 'r') as f:
    reader = csv.reader(f)
    header = next(reader)    # skip the header row
    print('Columns:', header)
    for row in reader:
        name, age, sal, dept = row
        print(f'{name} works in {dept}')
```

Using DictReader / DictWriter

```
# DictReader gives each row as a dictionary - easier to use!
with open('employees.csv', 'r') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['Name'], row['Salary'])

# DictWriter - write dicts to CSV
fields = ['Name', 'Age', 'Salary', 'Dept']
rows = [{'Name': 'Dave', 'Age': 28, 'Salary': 72000, 'Dept': 'Finance'}]
```

```
with open('new_emp.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerows(rows)
```

CSV with Pandas (Recommended)

```
import pandas as pd

# Read CSV into DataFrame
df = pd.read_csv('employees.csv')
print(df.head())

# Useful parameters
df = pd.read_csv('data.csv',
    sep=';',          # delimiter other than comma
    header=0,         # row number of header
    index_col='Name', # use 'Name' column as index
    usecols=['Name', 'Salary'], # load only these columns
    dtype={'Salary': float},    # force data type
    na_values=['N/A', '-'],     # treat these as NaN
    encoding='utf-8')

# Save DataFrame back to CSV
df.to_csv('output.csv', index=False) # index=False omits row numbers
```

5.3 JSON Files

JSON (JavaScript Object Notation) is the standard format for APIs and configuration files. Python's built-in `json` module makes it easy to read and write.

```
import json

# Python object to save
data = {
    'employees': [
        {'name': 'Alice', 'age': 25, 'salary': 50000},
        {'name': 'Bob', 'age': 30, 'salary': 65000},
    ],
    'company': 'TechCorp',
    'year': 2024
}

# ■■ Write JSON to file ■■
with open('data.json', 'w') as f:
    json.dump(data, f, indent=4) # indent makes it human-readable

# ■■ Read JSON from file ■■
with open('data.json', 'r') as f:
    loaded = json.load(f)

print(loaded['company']) # TechCorp
print(loaded['employees'][0]['name']) # Alice

# ■■ JSON string (not file) ■■
json_str = json.dumps(data) # dict → string
obj = json.loads(json_str) # string → dict
```

Reading JSON API data with Pandas

```
import pandas as pd, json

with open('data.json', 'r') as f:
    raw = json.load(f)

# Normalise nested JSON into a flat DataFrame
df = pd.json_normalize(raw['employees'])
print(df)

# Output:
#   name  age  salary
# 0  Alice   25   50000
# 1   Bob   30   65000
```

5.4 Excel Files

```
import pandas as pd

# Read Excel - requires openpyxl installed
df = pd.read_excel('sales.xlsx',
```

```

        sheet_name='Q1',    # or 0 for first sheet
        header=0)

# Read all sheets at once
all_sheets = pd.read_excel('sales.xlsx', sheet_name=None)
# all_sheets is a dict: {sheet_name: DataFrame}

# Write to Excel
df.to_excel('output.xlsx', sheet_name='Results', index=False)

# Write multiple sheets
with pd.ExcelWriter('report.xlsx', engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Data',    index=False)
    df.describe().to_excel(writer, sheet_name='Summary')

```

5.5 Binary Files & Pickle

```

import pickle

# Save any Python object (model, dict, list...) to binary
model_params = {'weights': [0.3, 0.7], 'bias': 0.1}

with open('model.pkl', 'wb') as f:
    pickle.dump(model_params, f)

# Load it back
with open('model.pkl', 'rb') as f:
    loaded_params = pickle.load(f)

print(loaded_params)    # {'weights': [0.3, 0.7], 'bias': 0.1}

# Pandas DataFrames can also be pickled
df.to_pickle('dataframe.pkl')
df2 = pd.read_pickle('dataframe.pkl')

```

■ *Pickle files are Python-specific. For sharing data with other languages use CSV or JSON. Never unpickle files from untrusted sources—they can execute arbitrary code.*

5.6 os and pathlib – Working with the File System

Before reading or writing files you often need to check existence, list directories, or construct paths. The `os` and `pathlib` modules handle this cleanly.

os Module

```

import os

# Current working directory
print(os.getcwd())          # /home/user/project

# List all files in a directory
files = os.listdir('data/')  # ['sales.csv', 'info.json']

# Check existence
os.path.exists('data/sales.csv')  # True / False
os.path.isfile('data/sales.csv')  # True if it's a file

```

```
os.path.isdir('data/')          # True if it's a directory

# Create directory (safe - no error if already exists)
os.makedirs('output/charts', exist_ok=True)

# Rename / delete
os.rename('old.csv', 'new.csv')
os.remove('temp.txt')           # delete a file

# Build path cross-platform
path = os.path.join('data', 'raw', 'sales.csv')
print(path)  # data/raw/sales.csv (or data\raw\sales.csv on Windows)
```

pathlib Module (Modern – Recommended)

```
from pathlib import Path

# Create a Path object
p = Path('data/sales.csv')

print(p.name)      # 'sales.csv'
print(p.stem)      # 'sales'
print(p.suffix)     # '.csv'
print(p.parent)    # data

# Check and create
p.exists()          # True / False
Path('output').mkdir(parents=True, exist_ok=True)

# Read / write text directly
text = p.read_text(encoding='utf-8')
p.write_text('Hello World', encoding='utf-8')

# Iterate over all CSV files in a folder
for csv_file in Path('data').glob('*.csv'):
    df = pd.read_csv(csv_file)
    print(f'Loaded {csv_file.name}: {len(df)} rows')

# Combine paths with / operator
base = Path('project')
output = base / 'results' / 'summary.csv'
print(output)  # project/results/summary.csv
```

6. Data Cleaning

Handling Missing Values

```
import pandas as pd, numpy as np

df = pd.DataFrame({
    'Name':    ['Alice', 'Bob', None, 'Dave'],
    'Age':     [25,      None, 35,    28],
    'Salary':  [50000,   65000, np.nan, 72000]
})

df.isnull().sum()          # count NaNs per column
df.dropna()                # drop rows with ANY missing value
df.dropna(subset=['Age'])  # drop rows where Age is missing only

# Fill missing values
df['Age'].fillna(df['Age'].mean(), inplace=True)  # fill with mean
df['Name'].fillna('Unknown', inplace=True)       # fill with string
df.fillna(method='ffill', inplace=True)         # forward fill
```

Handling Duplicates

```
df.duplicated().sum()      # number of duplicate rows
df.drop_duplicates(inplace=True)  # remove duplicates
df.drop_duplicates(subset=['Name'])  # based on one column
```

Data Type Conversion

```
df['Age']    = df['Age'].astype(int)          # float → int
df['Salary'] = df['Salary'].astype(float)
df['Date']   = pd.to_datetime(df['Date'])     # string → datetime

# Extract date parts
df['Year']   = df['Date'].dt.year
df['Month']  = df['Date'].dt.month
df['Day']    = df['Date'].dt.day
```

7. Matplotlib – Data Visualisation

```
import matplotlib.pyplot as plt
import numpy as np
```

Line Plot

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
revenue = [12000, 15000, 13500, 17000, 16000, 19000]

plt.figure(figsize=(8, 4))
plt.plot(months, revenue, marker='o', color='steelblue', linewidth=2)
plt.title('Monthly Revenue 2024')
plt.xlabel('Month'); plt.ylabel('Revenue (INR)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('revenue.png', dpi=150) # save as image
plt.show()
```

Bar Chart

```
depts = ['HR', 'IT', 'Finance', 'Sales']
avg_sal = [52000, 78000, 71000, 65000]

plt.bar(depts, avg_sal, color=['#7c3aed', '#0ea5e9', '#059669', '#d97706'])
plt.title('Average Salary by Department')
plt.ylabel('Salary (INR)')
plt.tight_layout(); plt.show()
```

Histogram

```
ages = np.random.randint(22, 60, size=200)
plt.hist(ages, bins=15, color='steelblue', edgecolor='white')
plt.title('Age Distribution of Employees')
plt.xlabel('Age'); plt.ylabel('Frequency')
plt.show()
```

Scatter Plot

```
exp = np.array([1, 2, 3, 5, 7, 10, 12, 15])
salary = np.array([30000, 35000, 42000, 55000,
                  68000, 85000, 95000, 110000])

plt.scatter(exp, salary, color='purple', s=80, alpha=0.7)
plt.title('Experience vs Salary')
plt.xlabel('Years of Experience'); plt.ylabel('Salary')
plt.show()
```

Subplots – Multiple Charts


```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Left chart
axes[0].plot(months, revenue, 'o-', color='steelblue')
axes[0].set_title('Revenue Trend')

# Right chart
axes[1].bar(depts, avg_sal, color='purple')
axes[1].set_title('Salary by Dept')

plt.tight_layout()
plt.savefig('dashboard.png', dpi=150)
plt.show()
```

8. Quick Reference Card

NumPy

```
np.array([1,2,3])
np.zeros((m,n))
np.arange(start,stop,step)
arr.reshape(r,c)
np.mean/std/sum/min/max(arr)
```

Pandas

```
pd.read_csv('f.csv')
df.head() / df.info()
df[df['col'] > x]
df.groupby('col').agg()
pd.merge(df1,df2,on='key')
```

File I/O

```
open('f.txt','r/w/a')
csv.reader/writer(f)
json.load(f) / json.dump(obj,f)
pd.read_csv / to_csv
pd.read_excel / to_excel
```

pathlib

```
Path('dir/file.csv')
p.exists() / p.mkdir()
Path('a') / 'b' / 'c.csv'
p.glob('*.csv')
p.read_text() / write_text()
```

Matplotlib

```
plt.plot(x,y)
plt.bar(cats,vals)
plt.hist(arr,bins=n)
plt.scatter(x,y)
plt.savefig('out.png',dpi=150)
```

Cleaning

```
df.isnull().sum()  
df.dropna()  
df.fillna(val)  
df.drop_duplicates()  
df['col'].astype(dtype)
```